# Finding Intruder Knowledge with Cap-Matching ⋆

Erin Hanna[1], Christopher Lynch[2], David Jaz Myers[3], and Corey Richardson[2]

[1] Department of Mathematics, Eastern University, USA
[2] Department of Computer Science, Clarkson University, USA
[3] Department of Mathematics, Oberlin College, USA

**Abstract.** Given two terms $s$ and $t$, a substitution $\sigma$ matches $s$ onto $t$ if $s\sigma = t$. We extend the matching problem to handle **Cap**-terms, which are constructed of function symbols from the signature and a **Cap** operator which represents an unbounded number of applications of function symbols from the signature to a set of **Cap**-terms. A **Cap**-term represents an infinite number of terms. A **Cap**-substitution maps variables to **Cap**-terms and represents an infinite number of term substitutions. **Cap** matching is the problem of, given a term $s$ and a **Cap**-term $T$, find a set of **Cap**-substitutions which represents the set of substitutions that matches $s$ onto all the terms $t$ represented by $T$. We give a sound, complete and terminating algorithm for **Cap**-matching, which has been implemented in Maude. We show how the **Cap**-matching problem can be used to find all the messages learnable by an active intruder in a cryptographic protocol, where the **Cap** operator represents all the possible functions that can be performed by the intruder.

## 1 Introduction

Matching is a key operation in such areas as automated theorem proving [11] and cryptographic protocol analysis [8]. It is the problem of finding an instantiation to make one term equal to another term. More specifically, given terms $s$ and $t$, the *matching problem* is to find a substitution $\sigma$ applied to the variables in $s$ to make $s\sigma$ identical to $t$ [3]. We can call $s$ a *pattern*, and call $t$ the *object*, and we want to find a way for the object to fit the pattern.

In this paper we extend the matching problem to another problem that we call **Cap**-matching. In **Cap**-matching the object is not a single object, but a representation of which infinitely many objects can be constructed. In other words, the object in the matching problem is now a pattern itself. The difference is that in the pattern $s$ given on the left hand side of the matching problem, the pattern is instantiated by adding symbols at the leaves of the tree representation of the pattern. Now the right hand side $t$ becomes a pattern where terms can be built by adding symbols in the interior or root of the tree.

We define ***Cap** terms* in this paper. A **Cap** term represents a set of ground terms, as opposed to one single term. **Cap** terms are built up as usual from symbols in the signature of the language. We also allow **Cap** terms to contain a union symbol, indicating the union of the set of terms that are represented. For example, if $a$ and $b$ are constant symbols then $a$ represents the set $\{a\}$, and $a \cup b$ represents the set $\{a, b\}$. In addition

⋆ This work was done with support from NSF grant DMS-1262737 during the summer of 2015.

to the union operator, we also allow a **Cap** operator, which is parameterized by a set of symbols from the language. The set of terms represented by a **Cap** term is the set of terms that can be constructed by adding those symbols on top of its arguments. For example, the **Cap** term $\mathbf{Cap}_{\{f\}}(a \cup b)$, where $f$ is a binary symbol, represents the infinite set of terms: $\{a, b, f(a,a), f(a,b), f(b,a), f(b,b), f(a,f(a,a)), f(a,f(a,b))$, etc.

**Cap** terms were first introduced in [2] to study a passive intruder in cryptographic protocol analysis. In that paper, a **Cap** could only appear once at the top of a term. In other words, it did not allow terms of the form of $g(\mathbf{Cap}(a \cup b))$ or $f(\mathbf{Cap}(a), \mathbf{Cap}(b))$. The problem studied there was of a similar structure, but they studied the word problem instead of the matching problem. In other words, in our definition given at the beginning, $s$ had to be a ground term. In that paper, they allowed equivalence modulo equational theories. There was also work on **Cap** terms in [1], which extended the work to unification problems. There, unification over a homomorphic theory was studied. But the intent in that paper was only to decide unification problems, not provide a representation of the set of all solutions.

The main result of this paper is that we have extended the terms to allow **Cap**s anywhere in the term. Also, instead of just solving a decision problem, we want to find a representation of all the (usually infinite) set of solutions of a **Cap** matching problem.

To be precise, given standard term $s$, possibly containing variables, and **Cap** term $T$, we give a set of inference rules to return a ***Cap** substitution*, which is a representation of a (usually infinite) set of solutions to the **Cap** matching problem. We prove that our method is sound, complete and terminating.

**Cap** terms can be seen as a simple tree automata [4]. In fact, [10] considers a tree automata formalism, which can be shown to contain **Cap** terms, and gives some decision results for them.

We apply our results to cryptographic protocol analysis. Given a description of a protocol, we consider an active intruder who is trying to attack that protocol, and an unbounded number of sessions of the protocol. We show how to apply **Cap** matching to generate a set of all the messages that an intruder could learn. Of course, this procedure might not halt, because this problem is undecidable. But when it does halt, we have a representative of the infinite amount of knowledge that the intruder could learn.

This is what distinguishes our work from most work on cryptographic protocol analysis. Most techniques are guaranteed to return a single attack on the protocol when they halt. Whereas our result gives a representative of all attacks. The only result we are aware of which does not concentrate on just one solution are [5] and [6], which use constraint solving techniques for a bounded number of sessions, which do not lose solutions. Their technique returns a set of constraints which is satisfied by all solutions.

We have focused our work on cryptographic protocol analysis, because the **Cap**s naturally represent abilities of the intruder to construct messages. Note that matching is all that is necessary to determine what the intruder can learn, so we don't need to perform full unification. However, even though we focus on cryptographic protocol analysis, we believe these techniques can also be useful in SMT [7] in software verification, where **Cap** terms can be used to represent an infinite model.

The inference rules given in this paper are an extension of standard inference rules for unification or matching [3]. We look at the top symbol of the left and right hand side of

the matching problem when deciding which inference rule to apply. When the top symbol is a function symbol then usual inference rules apply, but we need to introduce additional rules to handle the case when the right hand side symbol is a **Cap** symbol. We also need to handle the case when a single variable $x$ maps to two different values. Inference rules, such as Variable Elimination will not work with **Cap** terms, since solutions will be lost. So we use an intersection operator. Therefore, we must extend the notion of **Cap** term to allow the intersection operator. But then we provide a set of inference rules that remove the intersection operator, so the final solution is expressed only in terms of **Cap** terms without intersection.

The paper is structured as follows. We give our notation in Section 2, especially the notation of **Cap**s. In Section 3 we show how to apply this to cryptographic protocol analysis. In Section 4 we present the inference rules, plus a set of rewrite rules to remove intersections. In Section 5 we prove soundness, completeness and termination of the inference system. In Section 6 we conclude the paper and discuss future work and related work.

## 2 Definitions

In this section, we introduce the basic definitions. We will work over a fixed signature $\Sigma$, which is a set of function symbols with fixed arities. We will denote by $\Sigma_n$ the set of $n$-ary functions, and will refer to $\Sigma_0$ as the set of constants. We will assume that $\Sigma$ is finite.

Terms over $\Sigma$ with variables from the set $V$ are defined inductively to be

1. a constant $a \in \Sigma_0$, or a variable $x \in V$, or
2. $f(s_1, \ldots, s_n)$ where $f \in \Sigma_n$ and the $s_i$ are all terms.

We will always assume a finite set of variables $V$ which will contain only those variables used in the problems at hand. The set of terms over $\Sigma$ in variables from $V$ is denoted $\mathcal{T}(\Sigma, V)$. If there are no variables in play, so that $V$ is empty, then we abbreviate $\mathcal{T}(\Sigma, \emptyset)$ as $\mathcal{T}(\Sigma)$. The terms in this set are called *ground terms*.

A *substitution* $\sigma$ is a function $\sigma : V \to \mathcal{T}(\Sigma, W)$ from variables to terms. If $V = \{x_1, \ldots, x_n\}$ and $x_i\sigma = t_i$, then we will write $\sigma$ as $[x_1 \mapsto t_1, \ldots, x_n \mapsto t_n]$. We will identify $\sigma$ with its homomorphic extension to $T(\Sigma, V)$.

We introduce **Cap**-terms as a means of representing infinite sets of terms that can be inductively generated over subsets of the signature $\Sigma$.

**Definition 1.** *A* ***Cap***-term *over a signature $\Sigma$ is*

1. *a constant $a \in \Sigma_0$, or*
2. *$f(T_1, \ldots, T_n)$ for a function symbol $f \in \Sigma_n$, where the $T_i$ are* ***Cap****-terms, or*
3. *$T_1 \cup T_2$, where $T_1$ and $T_2$ are* ***Cap****-terms, or*
4. *$\mathbf{Cap}_\Gamma(T)$ for a non-empty subset of function symbols $\Gamma \subseteq \Sigma$, where $T$ is a* ***Cap****-term.*

*We denote the set of* ***Cap****-terms over a signature $\Sigma$ by $\mathcal{T}_{\mathbf{Cap}}(\Sigma)$.*

For example, $f(a, \mathbf{Cap}_{\Sigma}(b \cup c))$, and $g(\mathbf{Cap}_{\{h(\cdot)\}}(a))$ are **Cap**-terms over the signature $\Sigma = \{a, b, c, g(\cdot), h(\cdot), f(\cdot,\cdot)\}$. We think of a **Cap**-terms as representing sets of terms of a given shape. The above **Cap**-terms are thought of as representing "any term of the form $f(a,t)$, where $t$ is any term only having constants $b$ and $c$" and "any term of the form $g(h^n(a))$" respectively. This idea is made formal by the following definition.

**Definition 2.** *The interpretation $[\![T]\!]$ of a **Cap**-term $T$ is defined inductively as follows:*

1. $[\![a]\!] := \{a\}$ *for constant* $a \in \Sigma_0$,
2. $[\![f(T_1, \ldots, T_n)]\!] := \{f(s_1, \ldots, s_n) \mid s_i \in [\![T_i]\!]\}$ *for function symbol* $f \in \Sigma_n$,
3. $[\![T_1 \cup T_2]\!] := [\![T_1]\!] \cup [\![T_2]\!]$,
4. $[\![\mathbf{Cap}_{\Gamma}(T)]\!]$ *is the smallest set* $S$ *such that*
   (a) $[\![T]\!] \subseteq S$, *and*
   (b) *If* $s_1, \ldots, s_n$ *are in* $S$ *and* $f$ *is an* $n$*-ary function symbol in* $\Gamma$, *then* $f(s_1, \ldots, s_n)$ *is in* $S$.

A **Cap**-*substitution* is a function $\alpha : V \to \mathcal{T}_{\mathbf{Cap}}(\Sigma)$. As with usual substitutions, we identify $\alpha$ with its homomorphic extension to $\mathcal{T}(\Sigma, V)$.

**Definition 3.** *The interpretation $[\![\alpha]\!]$ of a **Cap**-substitution $\alpha = [x_1 \mapsto T_1, \ldots, x_n \mapsto T_n]$ is the set of all substitutions sending $x_i$ to elements of $[\![T_i]\!]$,*

$$[\![\alpha]\!] := \{[x_1 \mapsto t_1, \ldots, x_n \mapsto t_n] \mid t_i \in [\![T_i]\!]\}$$

Now we are ready to define **Cap**-matching problems and their solutions. A *constraint* is of the form $s \stackrel{?}{\in} T$ where $s$ is a term and $T$ is a **Cap**-term. A ***Cap**-matching problem* is a set of constraints, usually written as $\varphi_1, \cdots, \varphi_n$, where each $\varphi_i$ is a constraint. We say that a substitution $\tau$ is a *term-solution* of the **Cap**-matching problem $s_1 \in T_1, \ldots, s_n \in T_n$ when $s_i\tau \in [\![T_i]\!]$ for all $i$. We say that a **Cap**-substitution $\alpha$ is a **Cap**-*solution* of a **Cap**-matching problem $\Gamma$ when every $\sigma \in [\![\alpha]\!]$ is a term-solution of $\Gamma$. When the context is clear, we will just call $\tau$ and $\alpha$ solutions.

We will also consider constrained terms.

**Definition 4.** *Let $t$ be a term and $\varphi$ be a **Cap** matching problem. Then $t \mid \varphi$ represents the set of all $t\sigma$ such that that there is a solution $\alpha$ of $\varphi$ with $\sigma$ an element of $[\![\alpha]\!]$.*

A set $\{\alpha_1, \ldots, \alpha_n\}$ of **Cap**-solutions for a **Cap**-matching problem $\Gamma$ is called a *complete set of solutions* of $\Gamma$ if every term solution $\sigma$ is in some $[\![\alpha_i]\!]$. We will seek complete sets of solutions for **Cap**-matching problems.

For example, the problem $g(x) \stackrel{?}{\in} g(\mathbf{Cap}_{\{h(\cdot)\}}(a))$ is solved by the **Cap**-substitution $\alpha = [x \mapsto \mathbf{Cap}_{\{h(\cdot)\}}(a)]$ because every $\tau \in [\![\alpha]\!]$ is of the form $[x \mapsto h^n(a)]$ and $g(x)\tau = g(h^n(x)) \in [\![g(\mathbf{Cap}_{\{h(\cdot)\}}(a))]\!]$ always. Indeed, $\{\alpha\}$ is a complete set of solutions for this problem.

## 3 Application to Cryptographic Protocol Analysis

In this section we discuss how **Cap**-matching would be applied to cryptographic protocol analysis, since that is the main motivation of our work. The rest of the paper gives inference rules for solving **Cap**-matching, and shows those inference rules correct. This section is independent of those sections.

In cryptographic protocol analysis, we analyze the properties of a protocol in the presence of a malicious intruder. In the Dolev-Yao model, the intruder is an omnipresent, omnipotent entity that can view all messages sent on the network. Intruders come in two varieties, *passive* and *active*. Passive intruders do not send messages or otherwise participate in the protocol. Active intruders do send messages and have the ability to modify messages other participants send.

Cryptographic protocol analysis either considers the bounded case or the unbounded case. In the bounded case, a protocol is only run a specific number of times. In the unbounded case, there is no bound on the number of times a protocol can be run. We deal with the unbounded case. In the unbounded case it is undecidable to decide if an intruder can learn a secret.

Cryptographic protocols can be modeled as Horn clauses in first-order logic, where messages are represented by terms. Protocol rules correspond to definite clauses, or formulas of the form $P_1, \ldots, P_n \rightarrow C$, where $P_1, \cdots, P_n$ and $C$ are terms. This can be understood in the context of a protocol as a participant responding to the combination of messages $P_1$ through $P_n$ with $C$. Some or all of these messages could have been sent by an active intruder. Initial knowledge can be modeled as facts. A secret learned by an intruder is represented as a goal clause. Variables represent pieces of a message where a participant in the protocol will accept anything, such as a new nonce or an encrypted message which the participant cannot decrypt.

These clauses represent the protocol, and they are sufficient for a passive intruder. However, we are interested in modelling an active intruder. With an active intruder, additional clauses are needed to represent how an intruder can construct and destruct data. We give examples of constructors and destructors involving the pairing (concatenation) operator $p(m_1, m_2)$, where $m_1$ and $m_2$ are the messages to be concatenated, and the symmetric encryption operator $e(m, k)$, where $m$ is a message and $k$ is a secret key. Constructors for these operators are the following:

$$m_1, m_2 \rightarrow p(m_1, m_2)$$
$$m, k \rightarrow e(m, k)$$

This indicates that the intruder can always concatenate two known messages together. In addition the intruder can encrypt a message with a key if he knows the message and key.

Destructors for these operators are the following:

$$p(m_1, m_2) \rightarrow m_1$$
$$p(m_1, m_2) \rightarrow m_2$$
$$e(m, k), k \rightarrow m$$

This indicates that an intruder can take a concatenated message apart, and the intruder can decrypt an encrypted message if he knows the key that was used to encrypt it.

A cryptographic protocol analysis method that searches backwards from a secret to the initial knowledge will never halt if the secret is not derivable, because of the destructors. A method that searches forward from the initial knowledge to a secret will never halt if the secret is not derivable, because of the constructors.

Instead of just determining whether an intruder can learn a particular secret, we want to represent all of the knowledge that an intruder can learn. For that reason, we need to use forward search. That means we can leave the destructors as Horn clauses. However, we must deal with constructors in a different way. So we do not consider the constructors as Horn clauses, and we instead incorporate them into a matching procedure. That is the reason for developing **Cap**-matching.

Normally, forward search proceeds by starting with the initial facts and repeatedly matching known facts against the left side of a Horn clause to create new known facts from the right side of that Horn clause. The fixed point of this process gives all the knowledge known by the intruder. Because of the constructors, this process will never halt.

In our method, each initial fact $F$ is represented by the constrained term $F \mid \varphi$, where $F$ is term and $\varphi$ is a **Cap**-matching problem. We rename all the known constrained facts, so they do not share any variables with the other facts, and then we gather all the known facts $F_1 \mid \varphi_1, \ldots, F_m \mid \varphi_m$ together as a **Cap**-term $T = \mathbf{Cap}(F_1 \cup \cdots \cup F_m)$. and a **Cap**-matching problem $\varphi = \varphi_1, \cdots, \varphi_n$. Then, given a Horn Clause $P_1, \ldots, P_n \rightarrow C$, we form the **Cap**-matching problem $P_1 \stackrel{?}{\in} T, \ldots, P_n \stackrel{?}{\in} T, \varphi$. We run a **Cap**-matching algorithm on this problem. If this is solvable, the algorithm will return us a set of **Cap**-substitutions $\alpha_1, \ldots, \alpha_k$. This will give us $k$ new constrained facts $C \mid \alpha_1, \ldots, C \mid \alpha_k$. These facts are added to our known facts, and this process is repeated until a fixed point is reached.

Using constraints makes this a little more complicated than just applying the **Cap** substitutions. If, for each rule $P_1, \ldots, P_n \rightarrow C$, the right hand side $C$ does not have repeated variables, then it would be sound to apply the **Cap** substitution. But if $C$ has repeated variables, then we must save the solutions as constraints to preserve soundness. The reason for this is that suppose that $C$ was of the form $P(x, x)$ and some $\alpha_i$ was the **Cap**-substitution $[x \mapsto Cap(a \cup b]$. Then $P(x, x)\alpha_i$ would be $P(Cap(a \cup b), Cap(a \cup b))$. An instance of this is $P(a, b)$, but that is not an instance of $P(x, x)$.

## 4 Solving Cap-matching Problems

Here we will present a deduction system which will find a (non-empty) complete set of solutions to a **Cap**-matching problem when one exists, and will fail if not. We proceed in two phases: Matching and Reduction. In the Matching phase, we find all solutions. In the Reduction phase, we handle conflicts. The Matching phase is given by a series of deduction rules, while the Reduction phase is given by a series of rewrite rules. We begin with Matching.

As usual, to apply an inference rule, we don't care nondeterministically select a constraint in the **Cap**-matching problem that is an instantiation of the top of an inference

$$\frac{f(s_1,\ldots,s_n) \stackrel{?}{\in} g(T_1,\ldots,T_m)}{\textbf{Fail} \quad (\text{if } f \neq g)}\ \text{Clash}$$

$$\frac{f(s_1,\ldots,s_n) \stackrel{?}{\in} f(T_1,\ldots,T_n)}{s_1 \stackrel{?}{\in} T_1,\ldots,s_n \stackrel{?}{\in} T_n}\ \text{Decomp}$$

$$\frac{f(s_1,\ldots,s_n) \stackrel{?}{\in} \textbf{Cap}_F(T)}{s_1 \stackrel{?}{\in} \textbf{Cap}_F(T),\ldots,s_n \stackrel{?}{\in} \textbf{Cap}_F(T) \quad (\text{if } f \in F)}\ \textbf{Cap}\text{-Decomp 1}$$

$$\frac{f(s_1,\ldots,s_n) \stackrel{?}{\in} \textbf{Cap}_F(T)}{f(s_1,\ldots,s_n) \stackrel{?}{\in} T}\ \textbf{Cap}\text{-Decomp 2}$$

$$\frac{f(s_1,\ldots,s_n) \stackrel{?}{\in} S \cup T}{f(s_1,\ldots,s_n) \stackrel{?}{\in} S}\ \text{Split 1}$$

$$\frac{f(s_1,\ldots,s_n) \stackrel{?}{\in} S \cup T}{f(s_1,\ldots,s_n) \stackrel{?}{\in} T}\ \text{Split 2}$$

**Fig. 1.** Matching Phase rules

rule. We replace that constraint by the appropriate instantiation of the bottom of that inference rule.

We begin with a **Cap**-matching problem $\{s_1 \stackrel{?}{\in} T_1,\ldots,s_n \stackrel{?}{\in} T_n\}$, and saturate it under the inference rules. The **Clash** and **Decomp** rules are applied deterministically, because no other rule is applicable when they apply. The **Cap-Decomp** and **Split** rules are applied don't know nondeterminstically.

The Clash rule fails if we have $s \stackrel{?}{\in} T$, where $s$ and $T$ contain different root symbols, and then the entire branch fails. The Decomp rule decomposes $s \stackrel{?}{\in} T$ if they have the same root symbol. These are standard rules for matching problems which are unchanged for **Cap**-matching. The **Cap**-Decomp rule is new. This deals with something of the form $s \stackrel{?}{\in} T$ where $T$ has a **Cap** symbol at the root. In this case $s$ must either match one of the elements in the union that **Cap** is applied to, or else the top symbol of $s$ is a symbol that is introduced on the right hand side by **Cap**. Note that this breaks the problem into two branches. The Split rule handles the case where the right hand side has a union on the top, and it also breaks the problem into two branches.

The algorithm terminates either by failing, or by reaching a point where each branch contains only problems of the form $x \stackrel{?}{\in} T$. Denote by $F = \{B_1^f, \ldots, B_n^f\}$ the set of final branches. We would like each final branch $B_i^f$ to represent a **Cap**-solution to the original set of problems. If no variables were repeated in the original problem, then each final branch is of the form $\{x_1 \stackrel{?}{\in} T_1, \ldots, x_n \stackrel{?}{\in} T_n\}$, so that it represents the **Cap**-substitution $[x_1 \mapsto T_1, \ldots, x_n \mapsto T_n]$ as we wanted. But if the variables appear more than once, we may have $x \stackrel{?}{\in} S$ and $x \stackrel{?}{\in} T$ in the same branch. For a **Cap**-substitution $\alpha$ to solve this, we must have that $x\tau \in [\![S]\!] \cap [\![T]\!]$ for all $\tau \in [\![\alpha]\!]$. So $\alpha$ must map $x$ to a **Cap**-term $U$ such that $[\![U]\!] = [\![S]\!] \cap [\![T]\!]$. The goal of the Reduction phase is to find such a $U$.

At this point a standard matching procedure could easily solve the problem by checking if $S$ and $T$ are the same. It is more complicated in **Cap**-matching. $S$ and $T$ do not have to be the same. They just have to have some common instances. We need to find the common instances. But we must do it in such a way that $x$ is mapped to the common instances.

We begin by applying the following rule to the final branches as much as possible. This obviously halts, because each step reduces the number of constraints.

$$\frac{x \stackrel{?}{\in} T, x \stackrel{?}{\in} S, \Gamma}{x \stackrel{?}{\in} S \cap T, \Gamma} \text{ Merge}$$

The intersection symbol $\cap$ is not part of the definition of a **Cap**-term. So we have to push the $\cap$ symbol inside the term and finally remove it in order to first determine if there is a solution to this intersection, and then to represent this solution as a **Cap**-term.

So we use the following rewrite system to remove each $\cap$ to find a **Cap**-term which represents this intersection:

modulo $\cup$ and $\cap$ as AC symbols.

After reduction, each variable will either map to a $\Cap$-term containing no $\cap$ symbols and no $\emptyset$ symbols, or it will map to $\emptyset$. On a given branch, after reduction, if at least one variable maps to the empty set, then there is no solution, and this branch fails. If all variables map to a $\Cap$-term,, then there is guaranteed to be a solution. A solution can easily be found by examining the resulting $\Cap$-terms. In fact, it is easy to enumerate all solutions. We believe another benefit of this notation is that it is easy for a human to understand the notation and to recognize interesting solutions.

## 5 Proofs

In this section, we prove that both the Matching and Reduction phases are terminating, sound, and complete. The Matching phase algorithm consists in applying the four rules above to a **Cap**-matching problem non-deterministically. The algorithm terminates when every matching problem in the set is in solved form $x \stackrel{?}{\in} T$. No rule may be applied to a problem of this form, since the left hand side of every matching problem has a function symbol from $\Sigma$ on top. We quickly show that the Matching phase algorithm terminates.

**Proposition 1.** *The Matching phase algorithm terminates.*

$$f(S_1, \ldots, S_n) \cap f(T_1, \ldots, T_n) \to f(S_1 \cap T_1, \ldots, S_n \cap T_n) \tag{1}$$

$$f(S_1, \ldots, S_n) \cap \mathbf{Cap}_\Gamma(T) \to f(S_1, \ldots, S_n) \cap T \quad \text{if } f \notin \Gamma \tag{2}$$

$$f(S_1, \ldots, S_n) \cap \mathbf{Cap}_\Gamma(T) \to f(S_1 \cap \mathbf{Cap}_\Gamma(T), \ldots, S_n \cap \mathbf{Cap}_\Gamma(T)) \cup (f(S_1, \ldots, S_n) \cap T) \quad \text{if } f \in \Gamma \tag{3}$$

$$\mathbf{Cap}_\Gamma(S) \cap \mathbf{Cap}_\Delta(T) \to \mathbf{Cap}_{\Gamma \cap \Delta}((S \cap \mathbf{Cap}_\Delta(T)) \cup (\mathbf{Cap}_\Gamma(S) \cap T)) \tag{4}$$

$$S \cap (T_1 \cup T_2) \to (S \cap T_1) \cup (S \cap T_2) \tag{5}$$

$$\mathbf{Cap}_\emptyset(T) \to T \tag{6}$$

$$f(S_1, \ldots, S_n) \cap g(T_1, \ldots, T_m) \to \emptyset \quad \text{if } f \neq g \tag{7}$$

$$\mathbf{Cap}_\Gamma(\emptyset) \to \emptyset \tag{8}$$

$$f(S_1, \ldots, \emptyset, \ldots, S_n) \to \emptyset \tag{9}$$

$$T \cap \emptyset \to \emptyset \tag{10}$$

$$T \cup \emptyset \to T \tag{11}$$

**Fig. 2.** Reduction Phase rules

*Proof.* We provide a lexicographic ordering of terms and show that each rule decreases under this order. The order is $(l, r)$, where $l$ is the number of function symbols and **Cap**s on the left of the $\stackrel{?}{\in}$, and $r$ similarly on the right. Observe that each branch of each rule decreases under this order.

Now we can show that the Matching phase algorithm only produces solutions (is sound), and produces all solutions (is complete).

**Proposition 2.** *The Matching phase algorithm is sound and complete.*

*Proof.* We begin by proving soundness. We will show that if $\alpha$ is a **Cap**-substitution which solves the bottom of a rule, then it solves the top. The only solution of a terminal branch $\{x_1 \stackrel{?}{\in} T_1, \ldots, x_n \stackrel{?}{\in} T_n\}$ is $[x_1 \mapsto T_1, \ldots, x_n \mapsto T_n]$, so we will show that this solves the original set of problems by passing it up through every application of a rule.

Since $\alpha$ solves a problem whenever every $\tau \in [\![\alpha]\!]$ solves it, we will work with term substitutions directly.

**Decomp:** Suppose that $\tau$ solves the bottom of the **Decomp** rule. That is, suppose that $s_i\tau \in [\![T_i]\!]$ for all $i$. Then $f(s_1\tau, \ldots, s_n\tau) \in [\![f(T_1, \ldots, T_n)]\!]$, so that $\tau$ solves the top of the **Decomp** rule.
**Cap-Decomp:** We have two cases to consider. First suppose that $s_i\tau \in [\![\mathbf{Cap}_\Gamma(T)]\!]$ for all $i$. Then for any $f \in \Gamma$, $f(s_1\tau, \ldots, s_n\tau) = f(s_1, \ldots, s_n)\tau \in [\![\mathbf{Cap}_\Gamma(T)]\!]$ by the definition of $[\![\mathbf{Cap}_\Gamma(T)]\!]$. So $\tau$ solves the top.
Now suppose that $f(s_1, \ldots, s_n)\tau \in [\![T]\!]$. Then since $[\![T]\!] \subseteq [\![\mathbf{Cap}_\Gamma(T)]\!]$ by definition, $\tau$ solves the top.
**Split** If $s\tau \in [\![S]\!]$, then $s\tau \in [\![S]\!] \cup [\![T]\!] = [\![S \cup T]\!]$, and similarly if $s\tau \in [\![T]\!]$.

Now we turn to completeness. A **Cap**-matching problem may take the following forms:

$$x \stackrel{?}{\in} T \tag{1}$$

$$f(s_1, \ldots, s_n) \stackrel{?}{\in} g(T_1, \ldots, T_n) \tag{2}$$

$$f(s_1, \ldots, s_n) \stackrel{?}{\in} f(T_1, \ldots, T_n) \tag{3}$$

$$f(s_1, \ldots, s_n) \stackrel{?}{\in} \mathbf{Cap}_\Gamma(T) \tag{4}$$

$$f(s_1, \ldots, s_n) \stackrel{?}{\in} S \cup T \tag{5}$$

Possibilities (2)-(5) appear on the top of rules. We will show that if $\tau$ is a solution of any of these, then it is solution of the bottom of the respective rule. Since the algorithm terminates, any solution to any of these forms will also be a solution to a terminal branch of the algorithm, and will therefore have been produced by the algorithm.

1. This problem is in solved form.
2. There can be no solution to this problem since any term in the interpretation of the right hand side must have a $g$ on top, but the left has an $f$ on top. We may apply the **Clash** rule and rightly fail.
3. In this case, we may use the **Decomp** rule. If $\tau$ is a solution so that $f(s_1\tau, \ldots, s_n\tau) \in [\![f(T_1, \ldots, T_n)]\!]$, then $s_i\tau \in [\![T_i]\!]$ for all $i$ so that $\tau$ is a solution of the bottom.
4. In this case, we may use the **Cap-Decomp** rule 1 or 2. Suppose that $\tau$ is a solution so that $f(s_1, \ldots, s_n)\tau \in [\![\mathbf{Cap}_\Gamma(T)]\!]$. This could happen in two ways: either $f(s_1\tau, \ldots, s_n\tau)$ is in $[\![\mathbf{Cap}_\Gamma(T)]\!]$ because the $s_i\tau$ are in $[\![\mathbf{Cap}_\Gamma(T)]\!]$ and we capped them with $f \in \Gamma$, or $f(s_1, \ldots, s_n)\tau$ is in $[\![T]\!]$ directly. These are precisely the bottom of the **Cap-Decomp** rule 1 or 2.
5. In this case, we may use the **Split** rule 1 or 2. Suppose that $\tau$ is a solution so that $f(s_1, \ldots, s_i)\tau \in [\![S \cup T]\!]$. By the definition of set union, either $f(s_1, \ldots, s_i)\tau \in [\![S]\!]$ or $f(s_1, \ldots, s_i)\tau \in [\![U]\!]$. The **Split** rules explore both possibilities.

We have now shown that the solutions which solve a set of **Cap**-matching problems are precisely those which solve the terminal branches of the Matching phase algorithm applied to that set. But, these terminal branches may contain repeated variables, e.g. $x \in S$ and $x \in T$. The Reduction phase finds a **Cap**-term $U$ which represents the intersection $[\![S]\!] \cap [\![T]\!]$.

We begin by showing that the rewriting system given above is terminating.

**Proposition 3.** *The Reduction phase rewriting system is terminating.*

*Proof.* We provide a lexicographic path ordering. The precedence is $\cap \succ \mathbf{Cap} \succ \cup \succ \Sigma \succ \emptyset$, where all $\Sigma$-terms have equal precedence. We now show that the left hand side of every rule dominates the right.

- In rules (1), (3)-(6), the top symbol on the left precedes the top of the right. Therefore, we must compare the left to the subterms of the right. In (1), we see that $f(S_1, \ldots, S_n) \cap f(T_1, \ldots, T_n) > S_i \cap T_i$ since $S_i$ is a subterm of $f(S_1, \ldots, S_n)$, and similarly for $T_i$. In (3), consider the left argument of the right hand side. Since $\cap \succ \Sigma$, we need to show that the whole of the right dominates each $S_i \cap \mathbf{Cap}_\Gamma(T)$, which follows since $S_i$ is a subterm of $f(S_1, \ldots, S_n)$. The right argument of the right hand side is dominated because $T$ is a subterm of $\mathbf{Cap}_\Gamma(T)$. Rule (4) follows because $S$ is a subterm of $\mathbf{Cap}_\Gamma(S)$ and similarly for $T$. Rule (5) follows because $T_1$ and $T_2$ are subterms of $T_1 \cup T_2$, and rule (6) because the right is a subterm of the left.
- Rules (7)-(11) follow because all other symbols have precedence over $\emptyset$. Rule (2) follows since $T$ is a subterm of $\mathbf{Cap}_\Gamma(T)$.

Since many **Cap**-terms can have the same intersection, we do not consider the possible uniqueness of normal forms produced by this rewriting system. Every such normal form, however, contains no $\cap$ symbols.

**Proposition 4.** *The normal form of a* ***Cap****-term $S$ under the reduction system contains no $\cap$ symbols.*

*Proof.* We proceed by induction over the path ordering described above. The base case is if $S$ contains no $\cap$ symbols, and is trivial as no rewrite rule adds $\cap$ symbols unless one was already present. Suppose that each term dominated by a term $S$ will be rewritten to have no $\cap$ symbols. Recall that, in particular, $S$ dominates each of its proper subterms. If $S$ does not have a $\cap$ symbol on top, then we may rewrite its subterms separately so that they have no $\cap$ symbols by hypothesis so that $S$ may be rewritten to remove all $\cap$ symbols. Suppose instead that $S = T_1 \cap T_2$. Clearly, the left hand sides of rules (1)-(5), (7), and (10) exhaust the possible forms $T_1 \cap T_2$ may take, modulo reordering. Then, applying each rule, we arrive at a term dominated by $S$ which, by hypothesis, may be rewritten to remove all $\cap$ symbols.

**Proposition 5.** *The normal form of a* ***Cap****-term $S$ under the reduction system is either $\emptyset$ or contains no $\emptyset$ symbols.*

*Proof.* If a constraint contains $\emptyset$ in any context, except for by itself, then some rule removes it.

Furthermore, the rewrites of the Reduction phase do not change interpretation. To prove this, we first prove a tiny lemma.

**Lemma 1.** *If $f(s_1, \ldots, s_n) \in [\![\mathbf{Cap}_\Gamma(T)]\!]$ and $f \notin \Gamma$, then $f(s_1, \ldots, s_n) \in [\![T]\!]$.*

*Proof.* We could not have found $f(s_1, \ldots, s_n) \in [\![\mathbf{Cap}_\Gamma(T)]\!]$ by capping the $s_i$ with $f$, since $f$ is not in $\Gamma$. Therefore, $f(s_1, \ldots, s_n)$ must have been in $[\![T]\!]$ to begin with.

Now we can show that if $U$ is a normal form of the Reduction phase applied to $S \cap T$, then $[\![U]\!] = [\![S]\!] \cap [\![T]\!]$ as desired.

**Proposition 6.** *The Reduction phase rewrite system does not modify solution sets.*

*Proof.* We show that, for each rule, that the interpretation of the left and right hand sides are equal. For these purposes, we extend the definition of interpretation to include $[\![S \cap T]\!] = [\![S]\!] \cap [\![T]\!]$ and $[\![\emptyset]\!] = \emptyset$.

1. If $t \in [\![f(S_1, \ldots, S_n)]\!] \cap [\![f(T_1, \ldots, T_n)]\!]$, then $t$ has the form $f(s_1, \ldots, s_n)$ and $f(t_1, \ldots, t_n)$ for $s_i \in S_i$ and $t_i \in T_i$. Therefore, the $s_i$ must also be in $T_i$ and likewise the $t_i$ must be in the $S_i$, so that $t \in [\![f(S_1 \cap T_1, \ldots, S_n \cap T_n)]\!]$. The converse is routine.
2. If $t \in [\![f(S_1, \ldots, S_n)]\!] \cap [\![\mathbf{Cap}_\Gamma(T)]\!]$, and $f \notin \Gamma$, then $t$ has an $f$ on top, but this $f$ could not have been placed by the $\mathbf{Cap}_\Gamma$. Therefore, $t$ must be in $[\![T]\!]$. Since $[\![T]\!] \subseteq [\![\mathbf{Cap}_\Gamma(T)]\!]$, the converse holds as well.
3. If $t \in [\![f(S_1, \ldots, S_n)]\!] \cap [\![\mathbf{Cap}_\Gamma(T)]\!]$, and $f \in \Gamma$, then $t = f(s_1, \ldots, s_n)$ and either this $f$ was not placed by the $\mathbf{Cap}_\Gamma$, or it was. If it was not placed, then the reasoning from above holds. If it was placed, then $s_i \in [\![\mathbf{Cap}_\Gamma(T)]\!]$ for all $i$, so that $t \in [\![f(S_1 \cap \mathbf{Cap}_\Gamma(T), \ldots, S_n \cap \mathbf{Cap}_\Gamma(T))]\!]$. The converse is routine.
4. We will prove this inclusion by induction,

   $$[\![\mathbf{Cap}_\Gamma(S) \cap \mathbf{Cap}_\Delta(T)]\!] \subseteq [\![\mathbf{Cap}_{\Gamma\cap\Delta}((S \cap \mathbf{Cap}_\Delta(T)) \cup (T \cap \mathbf{Cap}_\Gamma(S)))]\!].$$

   Denote the left and right hand sides of this inequality by **LHS** and **RHS** respectively. Suppose that $t \in$ **LHS**. If $t$ is a constant symbol $a$, then by the above lemma, $t$ is in $[\![S]\!]$ and $[\![T]\!]$, whence it is in the right hand side. Now suppose that $t = f(s_1, \ldots, s_n) \in$ **RHS** and that if $s_i \in$ **RHS**, then $s_i \in$ **LHS** for induction. We have three (not necessarily disjoint) cases:
   (a) Suppose that $f \notin \Gamma$. Then $t \in [\![S]\!]$, so that it is in $[\![S]\!] \cap [\![\mathbf{Cap}_\Delta(T)]\!] \subseteq$ **RHS**.
   (b) Similarly, suppose that $f \notin \Delta$. Then $t \in [\![T]\!]$, so that it is in $[\![T]\!] \cap [\![\mathbf{Cap}_\Gamma(S)]\!] \subseteq$ **RHS**.
   (c) Finally, suppose that $f \in \Gamma \cap \Delta$. Now, $t$ may be in $[\![S]\!]$ or $[\![T]\!]$, in which case we use the arguments above. But if it is in neither, then it must have been reached by capping $s_1, \ldots, s_n$ with $f$. So $s_i$ must have been in $[\![\mathbf{Cap}_\Gamma(S) \cap \mathbf{Cap}_\Delta(T)]\!]$, whence by induction they are in **LHS**. Then we may cap them with $f$ on the left hand side, so that $f(s_1, \ldots, s_n) = t \in$ **LHS**.

   Now, going the other direction, we want to show that **RHS** $\subseteq$ **LHS**. Since $[\![S \cap \mathbf{Cap}_\Delta(T)]\!]$ and $[\![T \cap \mathbf{Cap}_\Gamma(S)]\!]$ are contained in **LHS**, capping them with any symbol from $\Gamma \cap \Delta$ remains in the **LHS**.
5. The rest are routine.

Taken all together, we have shown that the solutions $\tau$ of a given set of **Cap**-matching problems $\{s_1 \stackrel{?}{\in} T_1, \ldots, s_n \stackrel{?}{\in} T_n\}$ are precisely the solutions of the final branches $\{x_1 \stackrel{?}{\in} T_1, \ldots, x_n \stackrel{?}{\in} T_n\}$ of the Matching phase and the same branches with all duplicate variables removed by the reduction phase, $\{x_1 \stackrel{?}{\in} U_1, \ldots, x_n \stackrel{?}{\in} U_n\}$. The solutions to these last sets are precisely $[x_1 \mapsto U_1, \ldots, x_n \mapsto U_n]$. Therefore, the Matching and Reduction phases together give us all and only solutions to the original matching problems.

## 6 Conclusion

We have defined **Cap**-terms, which are an extension of standard terms. Terms are built from symbols in the signature as usual, but now they are allowed to contain **Cap** symbols, which represent an unbounded number of applications of function symbols to its arguments. A **Cap**-term represents infinitely many standard terms.

We have defined a new kind of matching called **Cap**-matching. A **Cap**-matching problem is of the form $s_1 \stackrel{?}{\in} T_1 \cdots, s_n \stackrel{?}{\in} T_n$, where the $s_i$ are standard terms and the $T_i$ are **Cap**-terms. The solution to a **Cap**-matching problem is a **Cap**-substitution, which potentially represents infinitely many standard substitutions.

The procedure has three phases. The first phase is a matching phase, which may result in a variable being mapped to more than one $\Cap$-term. The second makes variables map to only one $\Cap$-term by adding intersection symbols. The third phase rewrites the solutions to remove the intersection symbols. If the first phase fails then there is no solution. If it succeeds to then all branches give a potential $\Cap$-solution. In any solution, if the third phase results in some variable mapped to $\emptyset$ then that branch fails. Any branch that does not fail in that way gives a $\Cap$-term representing a possibly infinite set of solutions. Every $\Cap$-term represents at least one solution. The solutions are easy to enumerate, and we believe this format allows humans to easily find interesting solutions.

Our definitions of **Cap**-matching are motivated by cryptographic protocol analysis. Usual techniques in this area determine if an intruder can learn a secret. On the other hand, our methods create a representation of a possibly infinite set of all terms that an intruder can learn. If this is an infinite set, such a process would not halt if **Cap**-matching were not used.

There are several directions of future work that we plan to pursue. In an implementation of **Cap**-matching in cryptographic protocol analysis, redundancy checking is crucial. In other words, when a new fact is created, it must be checked whether this fact is subsumed by a previous fact. Without this check, the procedure will rarely halt. In order to check if newly generated $\Cap$-terms contain any new information, it is crucial to check if one **Cap**-term is contained in another one. We have already investigate ways of checking this by creating an algorithm to verify if one $\Cap$-term is a subset of another one.

Equational theories are used to model properties of cryptographic algorithms. So another line of research we have already started pursuing is to perform **Cap**-matching modulo an equational theory. This is difficult, but it would be useful for applications.

We think that cryptographic protocol analysis is not the only area where **Cap**-matching is useful. It could also be used in program verification. SMT procedures sometimes need decision procedures modulo a theory of Horn clauses. **Cap**-matching could be used to create decision procedures that halt. It would also give a representation of an infinite model, whereas SMT solvers currently deal with finite models.

We have considered Horn clauses in this paper. However there is no reason this method cannot be extended to **Cap**-resolution with general clauses. We have begun research in this area.

## References


1. Siva Anantharaman, Hai Lin, Christopher Lynch, Paliath Narendran, and Michaël Rusinowitch. Cap unification: application to protocol security modulo homomorphic encryption. In *Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security, ASIACCS 2010, Beijing, China, April 13-16, 2010*, pages 192–203, 2010.
2. Siva Anantharaman, Paliath Narendran, and Michaël Rusinowitch. Intruders with caps. In *Term Rewriting and Applications, 18th International Conference, RTA 2007, Paris, France, June 26-28, 2007, Proceedings*, pages 20–35, 2007.
3. Franz Baader and Wayne Snyder. Unification theory. *Handbook of automated reasoning*, 1:445–532, 2001.
4. H. Comon, M. Dauchet, R. Gilleron, C. Löding, F. Jacquemard, D. Lugiez, S. Tison, and M. Tommasi. Tree automata techniques and applications. Available on: http://www.grappa.univ-lille3.fr/tata, 2007. release October, 12th 2007.
5. Hubert Comon-Lundh, Véronique Cortier, and Eugen Zalinescu. Deciding security properties for cryptographic protocols. application to key cycles. *ACM Trans. Comput. Log.*, 11(2), 2010.
6. Hubert Comon-Lundh, Stéphanie Delaune, and Jonathan Millen. Constraint solving techniques and enriching the model with equational theories. *Formal Models and Techniques for Analyzing Security Protocols*, 5:35–61, 2010.
7. Harald Ganzinger, George Hagen, Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. DPLL( T): fast decision procedures. In *Computer Aided Verification, 16th International Conference, CAV 2004, Boston, MA, USA, July 13-17, 2004, Proceedings*, pages 175–188, 2004.
8. Catherine Meadows. The NRL protocol analysis tool: A position paper. In *4th IEEE Computer Security Foundations Workshop - CSFW'91, Franconia, New Hampshire, USA, June 18-20, 1991, Proceedings*, page 227, 1991.
9. Paliath Narendran, Andrew M. Marshall, and Bibhu Mahapatra. On the complexity of the tiden-arnborg algorithm for unification modulo one-sided distributivity. In *Proceedings 24th International Workshop on Unification, UNIF 2010, Edinburgh, United Kingdom, 14th July 2010.*, pages 54–63, 2010.
10. Andreas Reuß and Helmut Seidl. Bottom-up tree automata with term constraints. In *Logic for Programming, Artificial Intelligence, and Reasoning - 17th International Conference, LPAR-17, Yogyakarta, Indonesia, October 10-15, 2010. Proceedings*, pages 581–593, 2010.
11. John Alan Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM (JACM)*, 12(1):23–41, 1965.